

Test de evaluación — Módulo 5 (10 preguntas tipo test)

1) Una automatización “low-code/no-code con IA” bien planteada, en el contexto del módulo, se entiende mejor como...

- a) Un proceso que conecta sistemas (correo, formularios, CRM, etc.) y usa la IA como un paso concreto (clasificar, extraer, resumir) con controles y rutas. 
- b) Un sistema que sustituye el diseño de procesos, porque la IA decide sola qué hacer en cada caso sin necesidad de reglas ni métricas.
- c) Un conjunto de chats “sueltos” que responden a usuarios, sin que exista trazabilidad, validación de datos ni salidas operativas medibles.

2) Caso: vas a automatizar la creación de tickets desde emails. El negocio quiere “rapidez” pero también evitar duplicados y costes innecesarios. ¿Qué decisión inicial suele ser más coherente con el módulo?

- a) Usar solo polling (consultas periódicas) aunque exista webhook, porque “así se controla mejor” sin configurar eventos.
- b) Usar ejecución por lotes una vez al día, aunque el usuario espere respuesta inmediata, y asumir que eso no afectará a la experiencia.
- c) Priorizar un disparo por evento (webhook si existe) y añadir control de duplicados + validación de entrada para que sea operable. 

3) En un contrato de datos (JSON) para un flujo de empresa, ¿cuál es la idea más correcta sobre “campos mínimos” y “campos opcionales”?

- a) Los campos opcionales deben bloquear la ejecución si faltan, porque el flujo debe ser perfecto para considerarse profesional.
- b) Si falta un campo mínimo, lo más eficiente es inventarlo o inferirlo para que el flujo “avance” y no se quede a medias.
- c) Los campos mínimos son los que, si faltan, obligan a ruta de “faltan datos/revisión”; los opcionales pueden ir como “no indicado/null” sin romper el flujo. 

4) Caso: un paso de IA debe extraer pedido_id, importe y fecha desde un email. A veces el email no incluye el pedido. ¿Qué salida y tratamiento encajan mejor con lo aprendido?

- a) Devolver pedido_id: “no indicado” (o null según estándar), validar la salida y enviar el caso a revisión/petición de datos, evitando que el flujo ejecute acciones críticas. 
- b) Estimar un pedido_id probable usando el asunto del email y seguir el flujo normal, ya que “la IA suele acertar por contexto”.
- c) Saltarse la validación de salida y continuar si al menos se ha obtenido importe, porque el ticket ya se puede crear igualmente.

5) Sobre listas (arrays) en automatización, ¿cuál es el riesgo operativo más típico cuando un paso devuelve muchos elementos (por ejemplo, 100 emails o 200 líneas de pedido)?

- a) Que la herramienta “comprima” automáticamente la lista y por tanto se pierda información, aunque el flujo no consuma más operaciones.
- b) Que se multipliquen ejecuciones/acciones (tickets, emails, llamadas) de forma no controlada, aumentando coste y riesgo si no filtras o limitas. ✓
- c) Que el sistema convierta la lista en un único texto y eso siempre mejore la calidad, porque el modelo tendrá más contexto en cada paso.

6) Caso: en Make, necesitas procesar 10 adjuntos uno a uno y después enviar un único email con un resumen final. ¿Qué combinación de componentes es la más adecuada según el módulo?

- a) Usar un agregador al principio para “unir” los adjuntos y luego iterar, porque así se simplifica el flujo y se evita el coste por bundle.
- b) Usar iteración controlada (Iterator) para tratar cada adjunto y después un Aggregator para recombinar resultados en un solo resumen final. ✓
- c) Evitar tanto Iterator como Aggregator y procesar “todo a la vez”, aunque no puedas rastrear qué adjunto causó el error.

7) En la terminología comparada Make versus n8n, ¿qué correspondencia es la más correcta para traducir el mismo modelo mental entre herramientas?

- a) Scenario (Make) equivale a Workflow (n8n), y Module (Make) equivale a Node (n8n), manteniendo la idea de “proceso” y “paso”. ✓
- b) Bundle (Make) equivale a Workflow (n8n), porque ambos son la “unidad” que circula entre componentes en cada ejecución.
- c) Router (Make) equivale a Scheduler (n8n), porque ambos “deciden” por dónde va el flujo y cuándo debe ejecutarse.

8) Caso: un flujo con HTTP empieza a fallar. Ves estos códigos en logs: 401, 429 y 5xx. ¿Qué interpretación y tratamiento es más coherente con la operación mínima?

- a) 401 suele apuntar a credenciales/token; 429 a límite de peticiones; 5xx a fallo del servicio: conviene reintento controlado con espera (backoff) y ruta de error con alertas. ✓
- b) 401 significa que el servidor está caído; 429 significa que falta un campo en el JSON; 5xx significa que hay que subir la temperatura del modelo.
- c) Todos los códigos se tratan igual: repetir la llamada sin esperar y sin alertas hasta que “alguna” ejecución salga bien.

9) Caso: estás defendiendo ante dirección la elección entre Make y n8n para un flujo con datos sensibles y necesidad de auditoría. ¿Qué forma de argumentar es más “profesional” según el módulo?

- a) Elegir la herramienta que tenga más conectores “de serie”, porque eso garantiza que el mantenimiento será siempre menor en cualquier escenario.
- b) Construir una matriz de decisión con criterios ponderados (seguridad/hosting, observabilidad, datos/listas, gobierno, coste/volumen...), explicitar trade-offs y proponer

mitigaciones. 

c) Decidir por preferencia personal del equipo (“nos gusta más”) y justificarlo con una demo bonita, sin KPIs, sin riesgos y sin plan de operación.

10) Para considerar que una automatización con IA está “lista para operar” (mínimo de producción), ¿qué conjunto de evidencias y controles encaja mejor con el módulo?

a) Tener una demo en directo que salga bien una vez, aunque no haya validación, logs, alertas, ni forma de reproducir la ejecución.

b) Disponer de prompts “creativos” y un diseño visual atractivo, aunque no exista contrato de entrada/salida ni ruta de fallback cuando falle.

c) Checklist de credenciales y validación, trazabilidad (logs/trace_id), alertas, control de cambios/export, y demo que muestre 1 caso OK y 1 fallo tratado. 